Graph Deep Learning for Time Series Processing

McGill University

Andrea Cini

March 10, 2025



Swiss National
Science Foundation

What is this about?



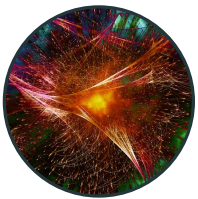
Traffic monitoring



Smart cities



Energy analitics



Physics



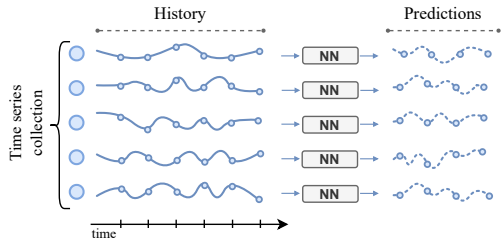
Stock markets

What is this about?

Deep learning for time series forecasting

Modern deep learning forecasting methods rely on a single neural network trained on a collection of related time series.

- ☺ Each time series is processed **independently**.
- ☺ Parameters are **shared**.
- ☺ Effective and **sample efficient**.
- ☹ **Dependencies are neglected**.



[1] Salinas *et al.*, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks", IJF 2020.

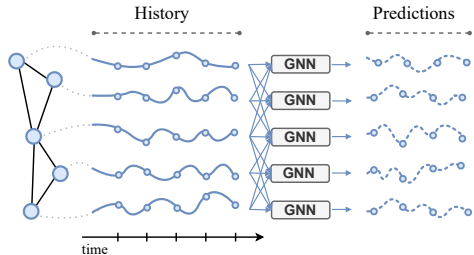
[2] Benidis *et al.*, "Deep Learning for Time Series Forecasting: Tutorial and Literature Survey", ACM CSUR 2022.

What is this about?

Graph deep learning for time series forecasting

Graph deep learning (GDL) provides appropriate operators to go beyond these limitations.

- ☺ **Dependencies** are **embedded into the processing** as inductive biases.
- ☺ Operate on **sets** of **correlated time series**.
- ☺ Parameters are **shared**.



☹ There are inherent **challenges** in applying this processing to data from the real world.

[3] **Cini** *et al.*, "Graph Deep Learning for Time Series Forecasting", preprint 2023.

What is this about?

Presentation outline

Part 1

1.1) Correlated time series

1.2) Forecasting models

1.3) Graph-based representation

1.4) STGNN architectures

Part 2

2.1) Local effects

2.2) Dealing with missing data

2.3) Scalability

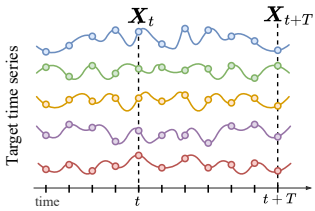
Conclusions

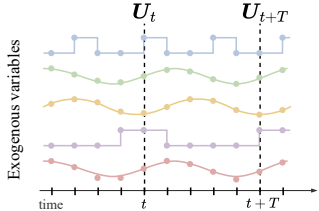
Forecasting correlated time series

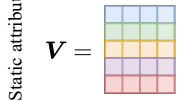
Collections of time series

Consider a set $\mathcal{D}$ of N correlated time series. Each i-th time series can be associated with:

- **observations** $\boldsymbol{x}_t^i \in \mathbb{R}^{d_x}$ at each time step t;
- **exogenous variables** $\boldsymbol{u}_t^i \in \mathbb{R}^{d_u}$ at each time step t;
- a vector of **static (time-independent) attributes** $\boldsymbol{v}^i \in \mathbb{R}^{d_v}$.







Capital letters denote the stacked N time series, i.e., $\boldsymbol{X}_t \in \mathbb{R}^{N \times d_x}$, $\boldsymbol{U}_t \in \mathbb{R}^{N \times d_u}$.

$\rightarrow$ We call spatial the dimension spanning the collection.

[3] **Cini** *et al.*, "Graph Deep Learning for Time Series Forecasting", preprint 2023.

Correlated time series

We can consider a time-invariant stochastic process generating each time series as

$$\boldsymbol{x}_t^i \sim p^i\left(\boldsymbol{x}_t^i \middle| \boldsymbol{X}_{<t}, \boldsymbol{U}_{\leq t}, \boldsymbol{V}\right) \quad \text{for all } i = 1 \dots N, t = 0, \dots, T-1$$

and assume the existence of a causality à la Granger among time series.

Furthermore time series

- are assumed
  a) homogenous, b) synchronous, c) regularly sampled.
- can be generated by different processes.

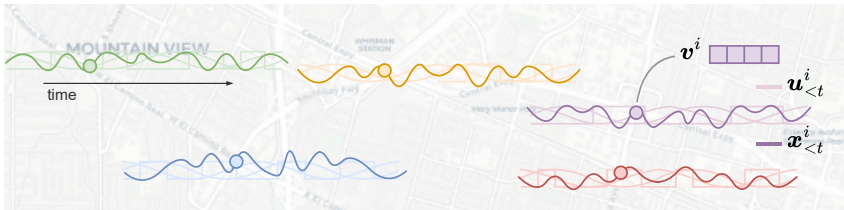
Notation:

$$\mathcal{X}_t = \langle \boldsymbol{X}_t, \boldsymbol{U}_t, \boldsymbol{V} \rangle$$
$$\mathcal{X}_{<t} = [\mathcal{X}_0, \cdots, \mathcal{X}_{t-2}, \mathcal{X}_{t-1}]$$

! Assumptions a),b),c) can be relaxed.

Example: Traffic monitoring system

Consider a sensor network monitoring the speed of vehicles at crossroads.



- $\boldsymbol{X}_{<t}$ collects past traffic speed measurements.
- $\boldsymbol{U}_t$ stores identifiers for time-of-the-day and day-of-the-week.
- $\boldsymbol{V}$ collects static sensor's features, e.g., type or number of lanes of the monitored road.

$\rightarrow$ Strong dependencies among time series that reflect the road network.

Forecasting models

Time series forecasting

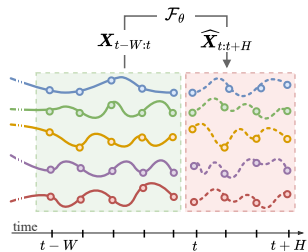
Multi-step time-series forecasting

Given a window of $W \geq 1$ past values

$$\mathcal{X}_{t-W:t} = [\mathcal{X}_{t-W}, \dots, \mathcal{X}_{t-1}],$$

predict $H \geq 1$ future observations

$$\boldsymbol{X}_{t+h} \qquad h = 1, \dots, H.$$



In particular, we are interested in learning a parametric model $\mathcal{F}(\,\cdot\,;\boldsymbol{\theta})$ s.t.

$$\mathcal{F}(\mathcal{X}_{t-W:t}, \boldsymbol{U}_{t:t+H}; \boldsymbol{\theta}) = \widehat{\boldsymbol{X}}_{t:t+H} \approx E_p\left[\boldsymbol{X}_{t:t+H}\right].$$

Probabilistic predictors can be considered as well, but we focus on point forecasts.

Training objective

For point predictors, parameters $\boldsymbol{\theta}$ can be learned by minimizing a cost function $\ell(\cdot, \cdot)$ (e.g., MSE) on a training set

$$\begin{aligned}\widehat{\boldsymbol{\theta}} &= \arg\min_{\boldsymbol{\theta}} \frac{1}{NT} \sum_{t=1}^{T} \ell\left(\widehat{\boldsymbol{X}}_{t:t+H}, \boldsymbol{X}_{t:t+H}\right) \\ &= \arg\min_{\boldsymbol{\theta}} \frac{1}{NT} \sum_{t=1}^{T} \left\| \boldsymbol{X}_{t:t+H} - \widehat{\boldsymbol{X}}_{t:t+H} \right\|_2^2.\end{aligned}$$

Global and local predictors

Local models



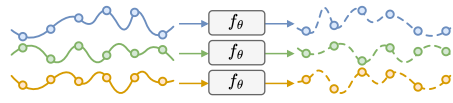
$$\hat{\boldsymbol{x}}^i_{t+h} = f\left(\boldsymbol{x}^i_{t-W:t}, \ldots; \boldsymbol{\theta}^i\right)$$

Example: Box-Jenkins method

☺ Tailored to each time series.

☹ Inefficient.

Global models



$$\hat{\boldsymbol{x}}^i_{t+h} = f\left(\boldsymbol{x}^i_{t-W:t}, \ldots; \boldsymbol{\theta}\right)$$

Example: DeepAR [1]

☺ Sample efficient.

☺ Allows for more complex models.

☹ Both approaches neglect dependencies among time series.

[1] Salinas *et al.*, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks", IJF 2020.

[4] Montero-Manso *et al.*, "Principles and algorithms for forecasting groups of time series: Locality and globality", IJF 2021.

Accounting for spatial dependencies

- One option is to consider the input as single multivariate time series
  - → Resulting predictors are **local**: $\widehat{\boldsymbol{X}}_{t+h} = f\left(\boldsymbol{X}_{t-W:t}, \dots; \boldsymbol{\theta}\right).$
    - ☹ Poor sample complexity and scalability.
- Models operating on sets of time series would allow to keep parameters shared.
  - → Resulting predictors are **global**: $\widehat{\boldsymbol{X}}^{\mathcal{S}}_{t+h} = \mathcal{F}\left(\boldsymbol{X}^{\mathcal{S}}_{t-W:t}, \dots; \boldsymbol{\theta}\right), \qquad \forall \mathcal{S} \subseteq \mathcal{D}$
    - ☺ Can be implemented by attention-based models (e.g, Transformers)
    - ☹ Does not exploit structural priors, high computational and sample complexity.
- Other methods (e.g., [5]) rely on dimensionality reduction to extract shared latent factors.
    - ☹ Local and relational information is lost and can still suffer from scalability issues.

[2] Benidis *et al.*, "Deep Learning for Time Series Forecasting: Tutorial and Literature Survey", ACM CSUR 2022.

[5] Sen *et al.*, "Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting", NeurIPS 2019.

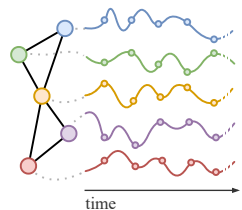
Graph-based forecasting

Relational information

Exploit functional dependencies as an inductive bias to improve the forecasts.

We can model pairwise relationships existing at time step t with adjacency matrix $\boldsymbol{A}_t \in \{0,1\}^{N \times N}$.

- $\boldsymbol{A}_t$ can be **asymmetric** and **dynamic** (can vary with t).



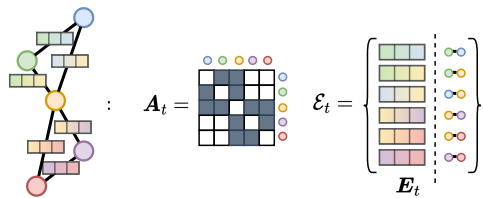
$\boldsymbol{A}_t =$

Relational information with attributes

Optional edge attributes $\boldsymbol{e}_t^{ij} \in \mathbb{R}^{d_e}$ can be associated to each non-zero entry of $\boldsymbol{A}_t$.

The set of attributed edges is denoted by

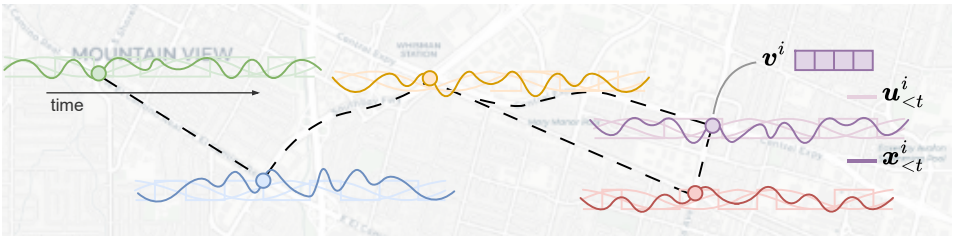
$$\mathcal{E}_t \doteq \{\langle (i,j), \boldsymbol{e}_t^{ij} \rangle \mid \forall i,j : \boldsymbol{A}_t[i,j] \neq 0\}.$$



$\rightarrow$ Edge attributes can be both **categorical** or **numerical**.

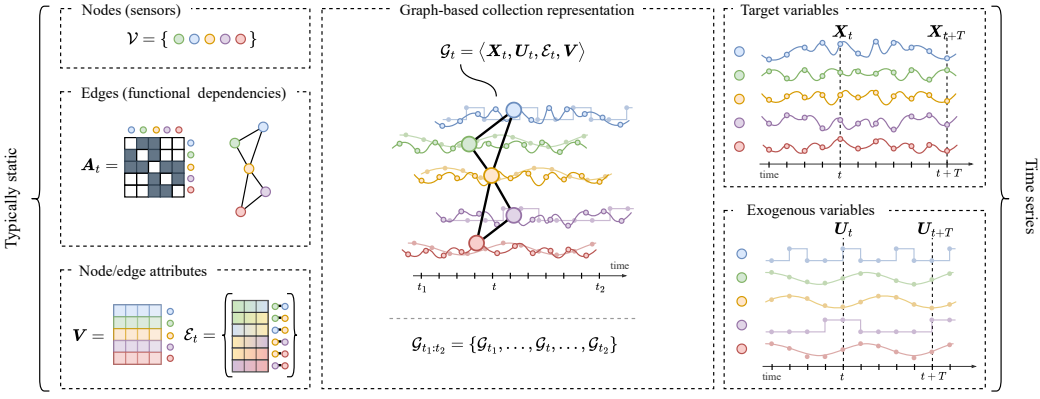
Example: Traffic monitoring system

Consider again the sensor network of the previous example.



- Edges in $\mathcal{E}$ can be obtained by considering the road network.
  - $\rightarrow$ Road closures and traffic diversions can be accounted for with a dynamic topology $\mathcal{E}_t$.

Graph-based representations for correlated time series



$\mathcal{G}_t \doteq \langle \boldsymbol{X}_t, \boldsymbol{U}_t, \mathcal{E}_t, \boldsymbol{V} \rangle$ contains the available information w.r.t. time step t.

[3] **Cini** *et al.*, "Graph Deep Learning for Time Series Forecasting", preprint 2023.

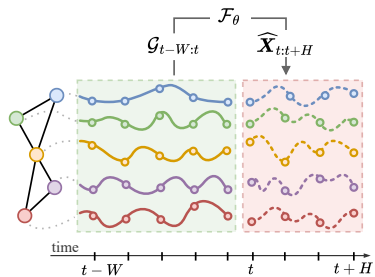
Relational inductive biases for time series forecasting

Forecasts can be conditioned on the available relational information $\mathcal{E}_{t-W:t}$

$$\widehat{\boldsymbol{X}}^{\mathcal{S}}_{t:T+H} = \mathcal{F}\left(\mathcal{G}^{\mathcal{S}}_{t-W:t}, \boldsymbol{U}^{\mathcal{S}}_{t:t+H}; \boldsymbol{\theta}\right) \qquad \forall \mathcal{S} \in \mathcal{D}$$

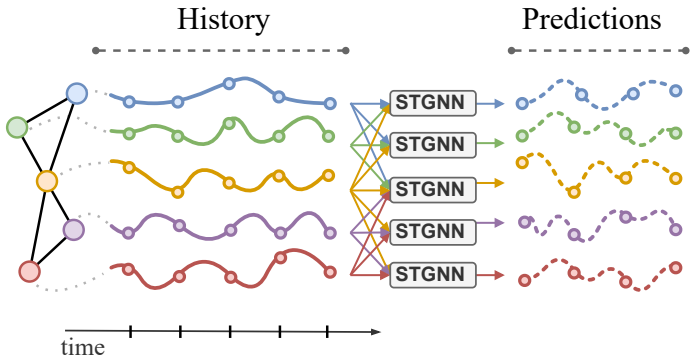
The conditioning can act as a regularization to localize predictions w.r.t. each node.

- ☺ Relational priors prune spurious correlations.
- ☺ More scalable than standard multivariate models.
- ☺ Can forecast any subset of correlated time series.



Spatiotemporal graph neural networks

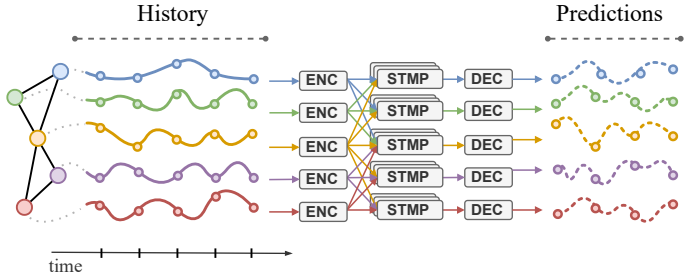
We call spatiotemporal graph neural networks (STGNNs) a neural network exploiting both temporal and spatial relations among input time series.



We focus on models based on message passing (MP).

A general recipe for building STGNNs

As a blueprint, we consider STGNNs consisting of three main components



- $\textsc{Enc}(\cdot)$ is the **encoding** layer, e.g., implemented by an MLP.
- $\textsc{STMP}(\cdot)$ is a stack of **spatiotemporal message-passing (STMP)** layers.
- $\textsc{Dec}(\cdot)$ is the **readout** layer, e.g., implemented by an MLP.

[3] **Cini** *et al.*, "Graph Deep Learning for Time Series Forecasting", preprint 2023.

Spatiotemporal message passing

STMP blocks can be defined as:

$$\boldsymbol{h}_t^{i,l+1} = \textsc{Up}^l\left(\boldsymbol{h}_{\leq t}^{i,l}, \underset{j \in \mathcal{N}_t(i)}{\textsc{Aggr}}\left\{\textsc{Msg}^l\left(\boldsymbol{h}_{\leq t}^{i,l}, \boldsymbol{h}_{\leq t}^{j,l}, \boldsymbol{e}_{\leq t}^{ji}\right)\right\}\right)$$

Each block processes **sequences** while accounting for **relational dependencies**.

As in standard MP operators:

- $\textsc{Msg}^l(\cdot)$ is a **message function**, e.g., implemented by *temporal convolutional layers*.
- $\textsc{Aggr}\{\cdot\}$ is a permutation invariant **aggregation function**.
- $\textsc{Up}^l(\cdot)$ is an **update function**, e.g., implemented by an RNN.

❗ Blocks can be implemented by composing MP and sequence modeling operators.

→ Many possible designs exist.

[3] **Cini** *et al.*, "Graph Deep Learning for Time Series Forecasting", preprint 2023.

[6] Gilmer *et al.*, "Neural message passing for quantum chemistry", ICML 2017.

Design paradigms for STGNNs

Depending on the implementation of the STMP blocks, we categorize STGNNs into:

- **Time-and-space (T&S)**
  Temporal and spatial processing cannot be factorized in two separate steps.
- **Time-then-space (TTS)**
  Each time series is embedded in a vector and then representations are propagated on the graph.
- **Space-then-time (STT)**
  Spatial propagation is performed before processing the resulting time series.

Time-and-space

In T&S models, representations at every node and time step are obtained by jointly propagating representation through time and space.

$$\boldsymbol{H}_{t-1}^{l+1} = \text{STMP}^{l}\Big(\boldsymbol{H}_{\leq t-1}^{l}, \mathcal{E}_{\leq t-1}\Big)$$

Several options exist.

- Integrate MP into neural operators for sequential data.
  - Graph recurrent architectures, spatiotemporal convolutions, spatiotemporal attention, ...
- Use sequence modelin operators to compute messages.
  - Temporal graph convolutions, spatiotemporal cross-attention, ...
- Product graph representations.

Example 1: From Recurrent Neural Networks...

Consider a standard GRU cell [7].

$$\boldsymbol{r}_t^i = \sigma\left(\boldsymbol{\Theta}_r \left[\boldsymbol{x}_t^i || \boldsymbol{h}_{t-1}^i\right] + \boldsymbol{b}_r\right) \tag{1}$$

$$\boldsymbol{u}_t^i = \sigma\left(\boldsymbol{\Theta}_u \left[\boldsymbol{x}_t^i || \boldsymbol{h}_{t-1}^i\right] + \boldsymbol{b}_u\right) \tag{2}$$

$$\boldsymbol{c}_t^i = \tanh\left(\boldsymbol{\Theta}_c \left[\boldsymbol{x}_t^i || \boldsymbol{r}_t^i \odot \boldsymbol{h}_{t-1}^i\right] + \boldsymbol{b}_c\right) \tag{3}$$

$$\boldsymbol{h}_t^i = \left(1 - \boldsymbol{u}_t^i\right) \odot \boldsymbol{c}_t^i + \boldsymbol{u}_t^i \odot \boldsymbol{h}_{t-1}^i \tag{4}$$

Time series are processed independently for each node or as a single multivariate time series.

[7] Chung *et al.*, "Empirical evaluation of gated recurrent neural networks on sequence modeling" 2014.

...to Graph Convolutional Recurrent Neural Networks

We can obtain a T&S model by implementing the gates of the GRU with MP blocks:

$$\boldsymbol{Z}_t^l = \boldsymbol{H}_t^{l-1} \tag{5}$$

$$\boldsymbol{R}_t^l = \sigma\left(\mathrm{MP}_r^l\left(\left[\boldsymbol{Z}_t^l || \boldsymbol{H}_{t-1}^l\right], \mathcal{E}_t\right)\right), \tag{6}$$

$$\boldsymbol{O}_t^l = \sigma\left(\mathrm{MP}_o^l\left(\left[\boldsymbol{Z}_t^l || \boldsymbol{H}_{t-1}^l\right], \mathcal{E}_t\right)\right), \tag{7}$$

$$\boldsymbol{C}_t^l = \tanh\left(\mathrm{MP}_c^l\left(\left[\boldsymbol{Z}_t^l || \boldsymbol{R}_t^l \odot \boldsymbol{H}_{t-1}^l\right], \mathcal{E}_t\right)\right), \tag{8}$$

$$\boldsymbol{H}_t^l = \boldsymbol{O}_t^l \odot \boldsymbol{H}_{t-1}^l + (1 - \boldsymbol{O}_t^l) \odot \boldsymbol{C}_t^l, \tag{9}$$

These T&S models are known as graph convolutional recurrent neural networks (GCRNNs) [8].

[8] Seo *et al.*, "Structured sequence modeling with graph convolutional recurrent networks", ICONIP 2018.

Popular GCRNNs

The first GCRNN has been introduced in [8], with message passing (MP) blocks implemented as polynomial graph convolutional filters.

GCRNNs have become popular in traffic forecasting with the Diffusion Convolutional Recurrent Neural Network (DCRNN) architecture [9].

DCRNN relies on a bidirectional diffusion convolution:

$$\boldsymbol{H}'_t = \sum_{k=0}^{K} \left(\boldsymbol{D}_{t,\text{out}}^{-1}\boldsymbol{A}_t\right)^k \boldsymbol{H}_t\boldsymbol{\Theta}_1^{(k)} + \left(\boldsymbol{D}_{t,\text{in}}^{-1}\boldsymbol{A}_t^\top\right)^k \boldsymbol{H}_t\boldsymbol{\Theta}_2^{(k)} \tag{10}$$

[8] Seo *et al.*, "Structured sequence modeling with graph convolutional recurrent networks", ICONIP 2018.

[9] Li *et al.*, "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting", ICLR 2018.

Example 2: Spatiotemporal convolutional networks (i)

Spatiotemporal convolutional networks (STCNs) instead **alternate spatial and temporal convolutions**:

1. Compute intermediate representations by using a temporal convolutional layer:

$$\boldsymbol{z}_{t-W:t}^{i,l} = \text{TCN}^l\left(\boldsymbol{h}_{t-W:t}^{i,l-1}\right) \qquad \forall\, i$$

   where TCN^l indicates a temporal convolutional layer.

2. Compute the updated representation at each time step by using a graph convolution:

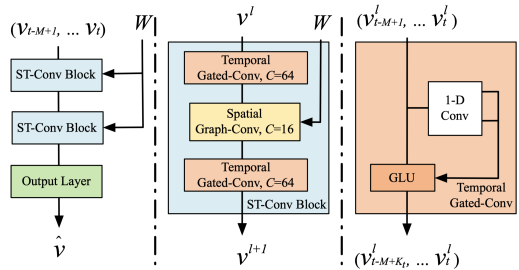
$$\boldsymbol{H}_t^l = \text{MP}^l\left(\boldsymbol{Z}_t^l, \mathcal{E}_t\right) \qquad \forall\, t$$

Spatiotemporal convolutional networks (ii)

The first example of such architecture is the STGCN by Yu et al. [10].

The model is obtained by stacking STMP blocks consisting of

- a (gated) temporal convolution;
- a polynomial graph convolution;
- a second (gated) temporal convolution.



Courtesy of [10].

More advanced implementations exist, e.g., see Graph Wavenet [11].

[10] Yu *et al.*, "Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting", IJCAI 2018.
[11] Wu *et al.*, "Graph wavenet for deep spatial-temporal graph modeling", IJCAI 2019.

Example 3: Temporal Graph Convolution

A more integrated approach instead consists of implementing a temporal propagation mechanism in the message function.

For example, we can design STMP layers s.t.

$$\boldsymbol{h}^{i,l}_{t-W:t} = \text{TCN}^l_1\left(\boldsymbol{h}^{i,l-1}_{t-W:t}, \underset{j\in\mathcal{N}_t(i)}{\text{AGGR}}\left\{\text{TCN}^l_2\left(\boldsymbol{h}^{i,l-1}_{t-W:t}, \boldsymbol{h}^{j,l-1}_{t-W:t}, \boldsymbol{e}^{ji}_{t-W:t}\right)\right\}\right).$$

Analogous models can be built with any sequence modeling architecture.

→ **Example:** many rely on attention-based operators [12][13].

[12] Marisca *et al.*, "Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations", NeurIPS 2022.

[13] Wu *et al.*, "TraverseNet: Unifying Space and Time in Message Passing for Traffic Forecasting", TNNLS 2022.

Example 4: Product graph representations

An alternative option is to consider the sequence $\mathcal{G}_{t-W:t}$ as a **single graph** with temporal and spatial edges.

In particular, **product graph representations** can be obtained by combining the two edge sets.

Temporal graph:



Spatial graph:

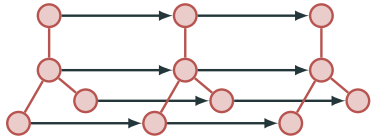


The resulting graph can be processed by any MP neural network.

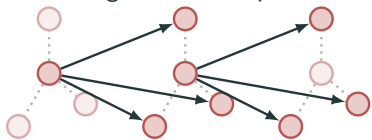
[14] Sabbaqi *et al.*, "Graph-time convolutional neural networks: Architecture and theoretical analysis", TPAMI 2023.

Building product graph representations

- **Cartesian product**
  Spatial graphs are kept and each node is connected to itself in the previous time instant.



- **Kronecker product**
  Each node is connected **only** to its neighbors in the previous time instant.



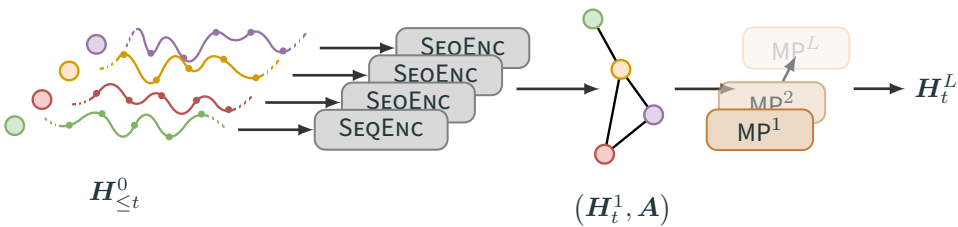
- ...

Time-then-Space models

The general recipe for a TTS model consists in:

1. Embedding each node-level time series in a vector.
2. Propagating obtained encodings throughout the graph with a stack of MP layers.

1. $\boldsymbol{h}_t^{i,1} = \textsc{SeqEnc}\left(\boldsymbol{h}_{\leq t}^{i,0}\right)$ 2. $\boldsymbol{H}_t^{l+1} = \text{MP}^l\left(\boldsymbol{H}_t^l, \mathcal{E}_t\right)$



Pros & Cons of TTS models

Pros:

- ☺ Easy to implement and computationally efficient.
- ☺ We can reuse operators we already know.

Cons:

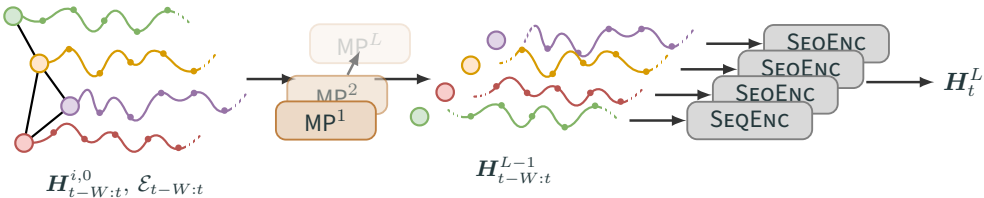
- ☹ The 2-step encoding might introduce information bottlenecks.
- ☹ Accounting for changes in topology and dynamic edge attributes can be more problematic.

Space-then-Time

In STT approaches the two processing steps of TTS models are inverted:

1. Observations are propagated among nodes w.r.t. each time step using a stack of MP layers.
2. Each sequence of representations is processed by a sequence encoder.

1. $\boldsymbol{H}_t^{i,l} = \text{MP}^l\left(\boldsymbol{H}_t^{i,l-1}, \mathcal{E}_t\right)$ 2. $\boldsymbol{h}_t^{i,L} = \textsc{SeqEnc}\left(\boldsymbol{h}_{t-W:t}^{i,L-1}\right)$

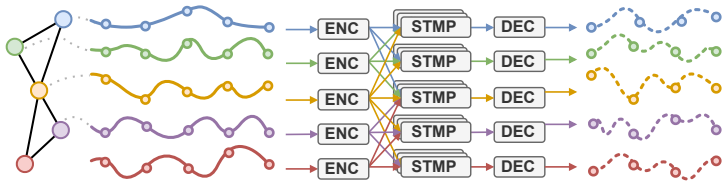


☹ They do not have the same computational advantages of TTS models.

Challenges

Ch. 1: Dealing with local dynamics

Globality and locality in STGNNs



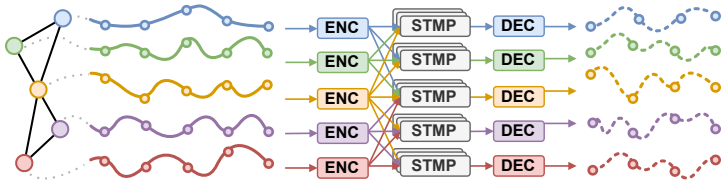
Standard STGNNs are **global** models.

- ☺ Can handle arbitrary node sets.
- ☺ Neighbors provide further conditioning on the predictions.
- ☹ Might struggle with local effects.
- ☹ Might need long windows and high model capacity.

Use hybrid global-local STGNNs.

[15] **Cini** *et al.*, "Taming Local Effects in Graph-based Spatiotemporal Forecasting", NeurIPS 2023.

Global-local STGNNs

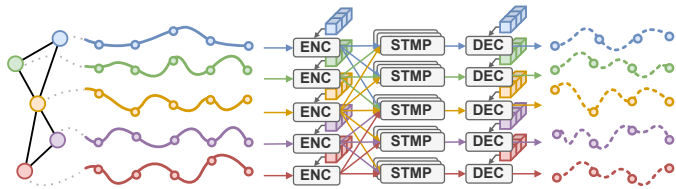


We can turn some global components of the architecture into local.

- Resulting models can capture local effects.
- Might require a large number of local parameters.

[15] **Cini** *et al.*, “Taming Local Effects in Graph-based Spatiotemporal Forecasting”, NeurIPS 2023.

Global-local STGNNs with node embeddings



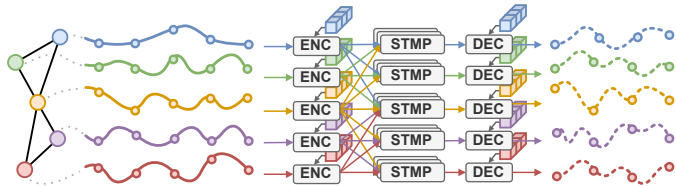
Node embeddings can amortize the learning of local components.

Node embeddings are a table of **learnable parameters** $Q \in \mathbb{R}^{N \times d_q}$ associated with **each node**.

- ☺ Fed into encoder/decoder, amortize the learning of local components.
- ☺ Most of the model's parameters remain shared.
- 😐 Number of parameters scales linearly with the number of time series . . .
  - → One might consider intermediate solutions, e.g., learning embeddings for clusters of time series.

[15] **Cini** *et al.*, "Taming Local Effects in Graph-based Spatiotemporal Forecasting", NeurIPS 2023.

Transferability



❗ Hybrid global-local STGNNs are not inductive models.

However, the cost of transfer learning can be reduced.

- ☺ Keep shared parameters fixed and finetune local parameters only.
- ☺ Node embeddings can be regularized to facilitate transfer further.

[15] **Cini** *et al.*, "Taming Local Effects in Graph-based Spatiotemporal Forecasting", NeurIPS 2023.

Empirical results on benchmark datasets

MODELS	MetrLA	PemsBAY	CER-E	AQI	MetrLA	PemsBAY	CER-E	AQI
Reference arch.	Global models				+ Local node embeddings			
RNN	3.54±.00	1.77±.00	4.57±.01	14.02±.04	3.15±.03	1.59±.00	4.22±.02	13.73±.04
GCRNN-IMP	3.35±.01	1.70±.01	4.44±.01	12.87±.02	3.10±.01	1.59±.00	4.18±.01	12.48±.03
RNN+IMP	3.34±.01	1.72±.00	4.39±.01	12.74±.02	3.08±.01	1.58±.00	4.12±.03	12.33±.02
GCRNN-AMP	3.22±.02	1.65±.00	4.57±.04	12.29±.02	3.07±.02	1.59±.00	4.17±.02	12.17±.05
RNN+AMP	3.24±.01	1.66±.00	4.31±.01	12.30±.02	3.06±.01	1.58±.01	4.13±.01	12.15±.02
Baseline arch.	Original				+ Local node embeddings			
DCRNN	3.22±.01	1.64±.00	4.28±.01	12.96±.03	3.07±.02	1.60±.00	4.13±.02	12.53±.02
GraphWaveNet	3.05±.03	**1.56±.01**	**3.97±.01**	12.08±.11	2.99±.02	1.58±.00	4.01±.01	11.81±.04
AGCRN	3.16±.01	**1.61±.00**	4.45±.01	13.33±.02	3.14±.00	1.62±.00	4.37±.02	13.28±.03

Table 1: MAE on benchmark datasets.

Transfer learning results

We consider datasets coming from four different traffic networks.

→ **One of the networks is left out** at training time and used for evaluating **transferability**.

	RNN+IMP	PEMS03	PEMS04	PEMS07	PEMS08
Fine-tuning	Global	15.30 ± 0.03	21.59 ± 0.11	23.82 ± 0.03	15.90 ± 0.07
	Embeddings	14.64 ± 0.05	20.27 ± 0.11	**22.23 ± 0.08**	**15.45 ± 0.06**
	– **Variational**	**14.56 ± 0.03**	20.19 ± 0.05	22.43 ± 0.02	**15.41 ± 0.06**
	– **Clustering**	**14.60 ± 0.02**	**19.91 ± 0.11**	**22.16 ± 0.07**	**15.41 ± 0.06**
Zero-shot		18.20 ± 0.09	23.88 ± 0.08	32.76 ± 0.69	20.41 ± 0.07

Table 2: Transfer learning results (MAE) after fine-tuning on a week of data.

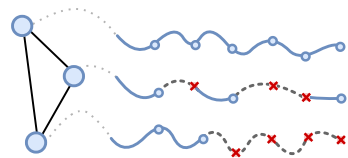
Ch. 2: Missing data

The problem of missing data

Time series collected in the real world often are incomplete and have missing data.

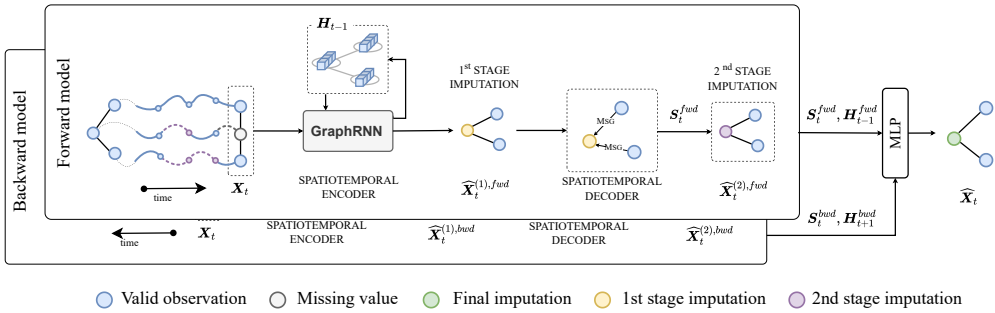
Most time series analysis tools operate on complete sequences.

- ⚠ We need a way to impute, i.e., *reconstruct*, missing data.
- ☺ We can exploit the available relational structure.



[16] **Cini** *et al.*, "Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks", ICLR 2022.
[12] Marisca, **Cini**, *et al.*, "Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations", NeurIPS 2022.
[17] De Felice, **Cini**, *et al.*, "Graph-based Virtual Sensing from Sparse and Partial Multivariate Observations", ICLR 2024.

Graph Recurrent Imputation Network (GRIN)

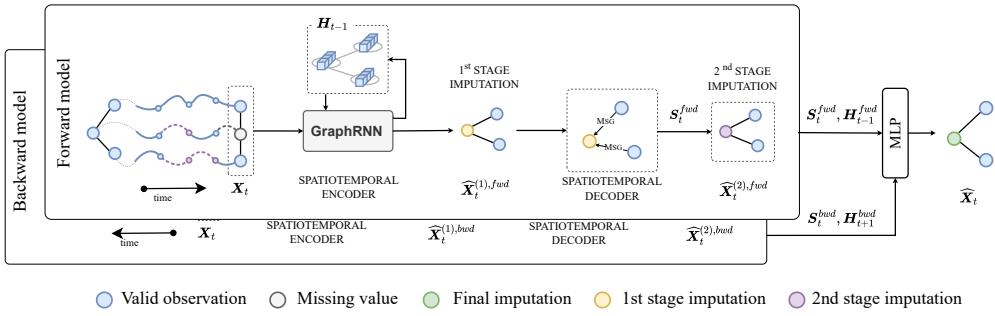


GRIN [16] is a bidirectional autoregressive STGNN.

- A **graph RNN** encodes the sequence of **valid observations**.
- An **final message-passing decoder** accounts for **concurrent observations at neighbors**

[16] **Cini** *et al.*, "Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks", ICLR 2022.

Limitations

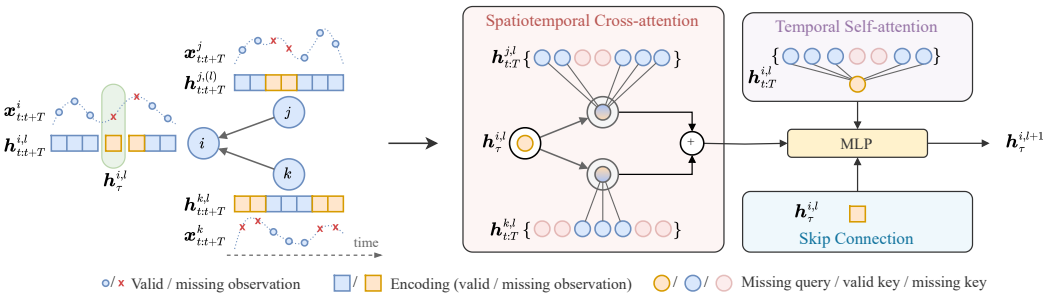


☺ Extremely effective in many settings.

☹ However, autoregressive models can accumulate reconstruction error.

[16] **Cini** *et al.*, "Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks", ICLR 2022.

Spatiotemporal Point Inference Network (SPIN)

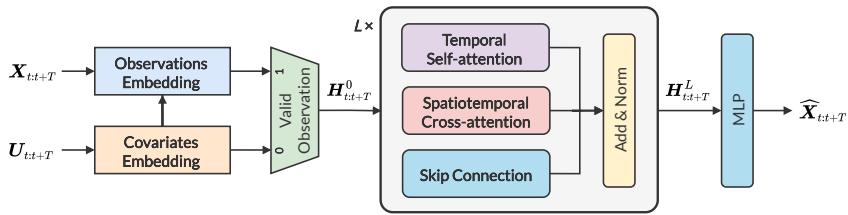


Use attention-based operators.

- We can exploit cross-attention to propagate information among related time series.
- Prevents autoregressive error accumulation.

[12] Marisca, **Cini**, *et al.*, "Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations", NeurIPS 2022.

Spatiotemporal Point Inference Network (SPIN)



- ☺ Effective even in scenarios with highly sparse observations.
- ☹ Can be computationally expensive.
  - → hierarchical computation can help by reducing the cost from $\mathcal{O}\left(T^2|\mathcal{E}|\right)$ to $\mathcal{O}\left(TK|\mathcal{E}|\right)$ ☺

[12] Marisca, **Cini**, *et al.*, "Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations", NeurIPS 2022.

Some empirical results

We compared the introduced **graph-based** methods against several state-of-the-art approaches including standard **attention-based** architectures and popular **RNN** models.

Model	Block missing		Point missing		Simulated failures	
	PEMS-BAY	METR-LA	PEMS-BAY	METR-LA	AQI-36	AQI
Mean	5.46±.00	7.48±.00	5.42±.00	7.56±.00	53.48±.00	39.60±.00
VAR	2.09±.10	3.11±.08	1.30±.00	2.69±.00	15.64±.08	22.95±.30
rGAIN	2.18±.01	2.90±.01	1.88±.02	2.83±.01	15.37±.26	21.78±.50
BRITS	1.70±.01	2.34±.01	1.47±.00	2.34±.00	14.50±.35	20.21±.22
SAITS	1.56±.01	2.30±.01	1.40±.03	2.26±.00	18.16±.42	21.33±.15
STTr	1.70±.02	3.54±.00	0.74±.00	2.16±.00	11.98±.53	18.11±.25
GRIN	1.14±.01	2.03±.00	**0.67±.00**	**1.91±.00**	12.08±.47	14.73±.15
SPIN	**1.06±.02**	**1.98±.01**	0.70±.01	**1.90±.01**	11.77±.54	**13.92±.15**
SPIN-H	**1.05±.01**	2.05±.02	0.73±.01	1.96±.03	**10.89±.27**	14.41±.13

Table 3: Reconstruction MAE.

Sensitivity analysis

Model	METR-LA			PEMS-BAY			AQI		
	Missing rate			Missing rate			Missing rate		
	50 %	75 %	95 %	50 %	75 %	95 %	50 %	75 %	95 %
BRITS	2.52±.00	3.02±.00	5.19±.02	1.55±.00	2.17±.00	3.91±.02	14.90±.03	18.29±.03	29.83±.07
SAITS	2.48±.00	3.74±.01	6.72±.01	1.50±.00	2.96±.01	7.40±.01	15.36±.02	20.64±.05	34.57±.05
STTr	2.31±.00	2.71±.00	5.13±.01	0.85±.00	1.13±.00	2.70±.01	9.11±.02	12.56±.05	25.65±.11
GRIN	2.05±.00	2.39±.00	4.08±.02	**0.79±.00**	1.09±.00	2.70±.01	8.43±.01	10.97±.02	20.38±.10
SPIN	2.02±.00	2.24±.00	2.89±.01	**0.79±.00**	1.00±.00	1.71±.00	**8.15±.01**	**9.96±.02**	**15.51±.08**
SPIN-H	**2.01±.00**	**2.20±.00**	**2.82±.00**	**0.79±.00**	**0.97±.00**	**1.68±.00**	8.67±.02	10.27±.02	15.75±.07

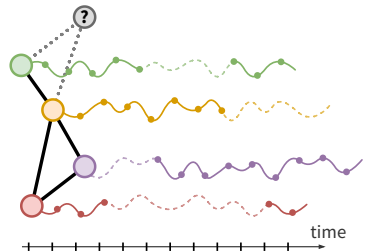
Table 4: MAE with increasing data sparsity in the *Point missing* settings.

Virtual sensing

Virtual sensing consists in estimating values at unmonitored locations

Graph-based models are particularly effective:

- ☺ We can condition estimates on neighboring sensors.
- ☺ Global models can handle unseen nodes and edges.
- ☺ Very useful when sensing is expensive.



[18] Wu *et al.*, "Inductive Graph Neural Networks for Spatiotemporal Kriging", AAAI 2021.
[17] De Felice, **Cini**, *et al.*, "Graph-based Virtual Sensing from Sparse and Partial Multivariate Observations", ICLR 2024.

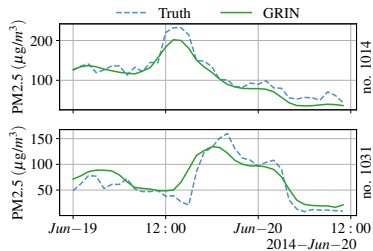
Graph-based virtual sensing

Add a virtual node with **no data** and let the model infer the corresponding time series.

Clearly, several assumptions are needed

- high degree of homogeneity of sensors,
- capability to reconstruct from observations at neighboring sensors,
- and many more...

Two virtual sensors for air quality. (from [16])



[16] **Cini** *et al.*, “Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks”, ICLR 2022.

[12] Marisca, **Cini**, *et al.*, “Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations”, NeurIPS 2022.

Ch. 3: Scalability

Computational scalability

Correlated time series must be processed along two dimensions:

Space – dimension corresponding to the number of time series (sensors);

& **Time** – dimension corresponding to the number of time steps (number of acquired observations).

Computational complexity of STGNNs

W: length of time series · N: n. of nodes · $|\mathcal{E}_t|$: n. of edges · L: n. of MP layers

The computational complexity of T&S models is given by:

- node-wise temporal processing – $\mathcal{O}(WN)$;
- L MP layers **for each time step** – $\mathcal{O}(WL|\mathcal{E}_t|)$.

$\rightarrow \mathcal{O}(W(N + L|\mathcal{E}_t|))$

TTS are more scalable:

- node-wise temporal processing – $\mathcal{O}(WN)$;
- L MP layers **at the last time step** – $\mathcal{O}(L|\mathcal{E}_t|)$.

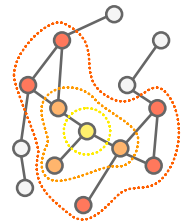
$\rightarrow \mathcal{O}(WN + L|\mathcal{E}_t|)$

There are two main approaches to mitigating this issue.

Graph subsampling

Computations can be reduced by training on subgraphs of the full network.

- sampling the K**-hop neighborhood** of a subset of nodes;
- **rewiring** the graph to reduce the total number of edges.



Mostly adapted from methods developed in **static graph processing**.

☹ Subsampling might break long-range spatiotemporal dependencies.

☹ The learning signal may be noisy.

[19] Gandhi *et al.*, "Spatio-Temporal Multi-graph Networks for Demand Forecasting in Online Marketplaces" 2021.
[20] Hamilton *et al.*, "Inductive representation learning on large graphs", NeurIPS 2017.
[21] Rong *et al.*, "DropEdge: Towards Deep Graph Convolutional Networks on Node Classification", ICLR 2020.

Pre-computation

Pre-processing methods (e.g., [22]) enable scalability to large graphs by:

- precomputing a representation for each node's neighborhood **ahead of training**;
- processing the obtained node representations as if they were **i.i.d. samples**.

An extension to spatiotemporal data is given by SGP [23], which acts in 2 steps:

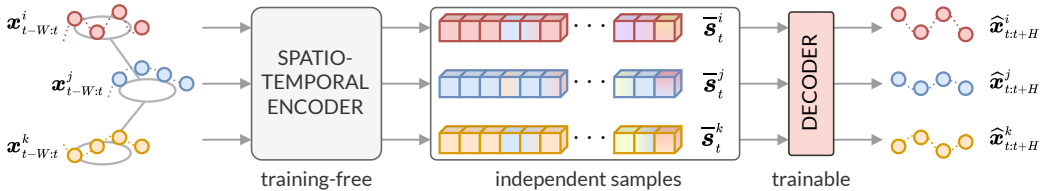
1. obtain a temporal encoding at each time step with a deep echo state network[1];
2. propagate such encodings through the graph using powers of a graph shift operator.

[22] Frasca *et al.*, "SIGN: Scalable inception graph neural networks" 2020.

[23] **Cini** *et al.*, "Scalable Spatiotemporal Graph Neural Networks", AAAI 2023.

[1] A randomized recurrent neural networks

SGP: Scalable Graph Predictor



SGP is based on an encoder-decoder architecture.

- ☺ The decoder is the only trainable component (can be a standard MLP).
- ☺ Any propagation through time and space happens at the encoding step.
- ☺ Encoded observations can then be sampled uniformly.
- ☺ The cost of a training step becomes independent of W and $|\mathcal{E}_t|$.

[23] **Cini** *et al.*, "Scalable Spatiotemporal Graph Neural Networks", AAAI 2023.

Empirical results

Models	METR-LA					PEMS-BAY				
	15 m	30 m	60 m	Average		15 m	30 m	60 m	Average	
	MAE	MAE	MAE	MAE	MAPE	MAE	MAE	MAE	MAE	MAPE
LSTM	2.99±.00	3.58±.00	4.43±.01	3.58±.00	1.19±.05	1.39±.00	1.83±.01	2.35±.01	1.79±.00	4.16±.05
FC-LSTM	3.33±.01	3.43±.01	3.67±.01	3.46±.01	1.15±.09	2.22±.01	2.25±.01	2.34±.02	2.26±.01	5.33±.04
DynGESN	3.27±.00	3.99±.00	5.00±.00	3.98±.00	11.11±.01	1.57±.00	2.13±.01	2.81±.02	2.09±.01	4.74±.01
DCRNN	2.82±.00	3.23±.01	3.74±.01	3.20±.00	8.88±.05	1.36±.00	1.71±.00	2.08±.01	1.66±.00	3.76±.01
GWNet	2.72±.01	3.10±.02	3.54±.03	3.06±.02	8.40±.03	1.31±.00	1.64±.01	1.94±.01	1.58±.00	3.58±.02
FC-GGN	2.72±.01	**3.05±.01**	**3.44±.01**	3.01±.00	**8.27±.00**	1.32±.00	1.63±.01	**1.89±.01**	1.56±.01	3.51±.03
UG-GGN	2.72±.00	3.10±.00	3.54±.01	3.06±.00	8.40±.04	1.33±.00	1.67±.01	1.99±.01	1.61±.01	3.59±.03
SGP	**2.69±.00**	**3.05±.00**	**3.45±.00**	**3.00±.00**	**8.27±.02**	**1.30±.00**	**1.60±.00**	**1.88±.00**	**1.54±.00**	**3.44±.01**

Table 5: Results on benchmark traffic datasets.

☺ SGP matches the state of the art in popular benchmarks . . .

Empirical results

	Models	Prediction error (MAE)			Resource utilization		
		30 mins	7 hours 30 mins	11 hours	Batch/s	Memory	Batch size
PV-US 100	DCRNN	1.39 ± 0.09	3.34 ± 0.22	**3.54 ± 0.48**	2.04 ± 0.01	9.63 GB	2
	GWNet	1.45 ± 0.13	5.09 ± 0.63	5.26 ± 1.34	2.01 ± 0.02	11.64 GB	2
	GatedGN	1.33 ± 0.08	**2.94 ± 0.05**	**3.12 ± 0.14**	8.41 ± 0.09	11.46 GB	5
	SGP	**1.09 ± 0.01**	**3.14 ± 0.21**	**3.16 ± 0.19**	**116.58 ± 8.74**	**2.21 GB**	4096
PV-US Full	DCRNN	1.59 ± 0.17	4.10 ± 0.27	4.93 ± 0.60	1.37 ± 0.00	11.59 GB	1*
	GWNet	1.65 ± 0.23	6.93 ± 0.58	7.93 ± 0.17	0.77 ± 0.00	11.35 GB	2
	GatedGN	1.61 ± 0.06	3.25 ± 0.04	**3.04 ± 0.05**	8.83 ± 0.10	11.14 GB	1*
	SGP	**1.09 ± 0.00**	**3.06 ± 0.11**	**3.13 ± 0.13**	**118.64 ± 8.35**	**2.21 GB**	4096

Table 6: Results on PV-US (5,016 nodes and from 420K (100-NN) to 3.7M (Full) edges).

. . . while having vastly lower memory consumption and higher throughput ☺

- ☹ Considerable amount of disk storage required for the pre-computed representations.
- ☹ More reliant on hyperparameter selection than end-to-end approaches.

Conclusions

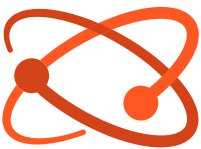
Some Takeaways

Deep Learning for **time series** + Deep Learning on **graphs**

- Relational inductive biases allow for exploiting dependencies among the time series,
- ... and deal with irregularities in time and space.
- Whenever possible, global-local models are a safe starting point.
- More challenges discussed in the full tutorial: graph learning, model quality assessment, . . .

Challenges. Local effects • Missing data • Scalability

tsl: Torch Spatiotemporal



tsl (Torch Spatiotemporal) is a python library built upon PyTorch and PyG to accelerate research on neural spatiotemporal data processing methods, with a focus on **Graph Neural Networks**.

torch-spatiotemporal.readthedocs.io

github.com/TorchSpatiotemporal/tsl

Notebook

Spatiotemporal Graph Neural Networks with tsl

Open in Colab

Contributors



Andrea **Cini**



Ivan **Marisca**

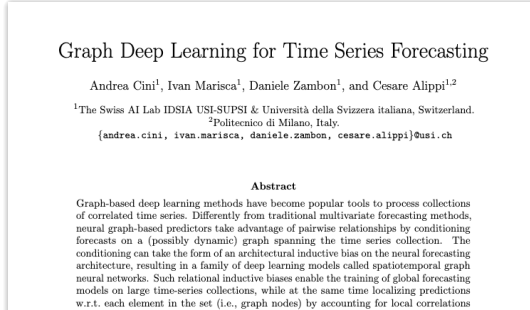


Daniele **Zambon**



Cesare **Alippi**





Graph Deep Learning for Time Series Forecasting

Andrea Cini[1], Ivan Marisca[1], Daniele Zambon[1], and Cesare Alippi[1,2]

[1]The Swiss AI Lab IDSIA USI-SUPSI & Università della Svizzera italiana, Switzerland.
[2]Politecnico di Milano, Italy.
{andrea.cini, ivan.marisca, daniele.zambon, cesare.alippi}@usi.ch

Abstract

Graph-based deep learning methods have become popular tools to process collections of correlated time series. Differently from traditional multivariate forecasting methods, neural graph-based predictors take advantage of pairwise relationships by conditioning forecasts on a (possibly dynamic) graph spanning the time series collection. The conditioning can take the form of an architectural inductive bias on the neural forecasting architecture, resulting in a family of deep learning models called spatiotemporal graph neural networks. Such relational inductive biases enable the training of global forecasting models on large time-series collections, while at the same time localizing predictions w.r.t. each element in the set (i.e., graph nodes) by accounting for local correlations

THE END

Questions?

References i

[1] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, **"DeepAR: Probabilistic forecasting with autoregressive recurrent networks,"** *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.

[2] K. Benidis, S. S. Rangapuram, V. Flunkert, *et al.*, **"Deep learning for time series forecasting: Tutorial and literature survey,"** *ACM Comput. Surv.*, vol. 55, no. 6, Dec. 2022, ISSN: 0360-0300. DOI: 10.1145/3533382. [Online]. Available: https://doi.org/10.1145/3533382.

[3] **A. Cini**, I. Marisca, D. Zambon, and C. Alippi, **"Graph deep learning for time series forecasting,"** *arXiv preprint arXiv:2310.15978*, 2023.

[4] P. Montero-Manso and R. J. Hyndman, **"Principles and algorithms for forecasting groups of time series: Locality and globality,"** *International Journal of Forecasting*, vol. 37, no. 4, pp. 1632–1653, 2021.

[5] R. Sen, H.-F. Yu, and I. S. Dhillon, **"Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting,"** *Advances in Neural Information Processing Systems*, vol. 32, 2019.

References ii

[6] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, **"Neural message passing for quantum chemistry,"** in *International conference on machine learning*, PMLR, 2017, pp. 1263–1272.

[7] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, **"Empirical evaluation of gated recurrent neural networks on sequence modeling,"** *arXiv preprint arXiv:1412.3555*, 2014.

[8] Y. Seo, M. Defferrard, P. Vandergheynst, and X. Bresson, **"Structured sequence modeling with graph convolutional recurrent networks,"** in *International Conference on Neural Information Processing*, Springer, 2018, pp. 362–373.

[9] Y. Li, R. Yu, C. Shahabi, and Y. Liu, **"Diffusion convolutional recurrent neural network: Data-driven traffic forecasting,"** in *International Conference on Learning Representations*, 2018.

[10] B. Yu, H. Yin, and Z. Zhu, **"Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,"** in *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, 2018, pp. 3634–3640.

References iii

[11] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, **"Graph wavenet for deep spatial-temporal graph modeling,"** in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2019, pp. 1907–1913.

[12] I. Marisca, **A. Cini**, and C. Alippi, **"Learning to reconstruct missing data from spatiotemporal graphs with sparse observations,"** in *Advances in Neural Information Processing Systems*, 2022.

[13] Z. Wu, D. Zheng, S. Pan, Q. Gan, G. Long, and G. Karypis, **"Traversenet: Unifying space and time in message passing for traffic forecasting,"** *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[14] M. Sabbaqi and E. Isufi, **"Graph-time convolutional neural networks: Architecture and theoretical analysis,"** *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 14 625–14 638, Dec. 2023, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2023.3311912.

[15] **A. Cini**, I. Marisca, D. Zambon, and C. Alippi, **"Taming local effects in graph-based spatiotemporal forecasting,"** *arXiv preprint arXiv:2302.04071*, 2023.

References iv

[16] **A. Cini**, I. Marisca, and C. Alippi, **"Filling the g_ap_s: Multivariate time series imputation by graph neural networks,"** in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=kOu3-S3wJ7.

[17] G. De Felice, **A. Cini**, D. Zambon, V. Gusev, and C. Alippi, **"Graph-based Virtual Sensing from Sparse and Partial Multivariate Observations,"** in *International Conference on Learning Representations*, 2024.

[18] Y. Wu, D. Zhuang, A. Labbe, and L. Sun, **"Inductive Graph Neural Networks for Spatiotemporal Kriging,"** in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 4478–4485.

[19] A. Gandhi, Aakanksha, S. Kaveri, and V. Chaoji, **"Spatio-temporal multi-graph networks for demand forecasting in online marketplaces,"** in *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part IV*, 2021, pp. 187–203, ISBN: 978-3-030-86513-9. DOI: 10.1007/978-3-030-86514-6_12. [Online]. Available: https://doi.org/10.1007/978-3-030-86514-6_12.

References v

[20] W. Hamilton, Z. Ying, and J. Leskovec, **"Inductive representation learning on large graphs,"** *Advances in neural information processing systems*, vol. 30, 2017.

[21] Y. Rong, W. Huang, T. Xu, and J. Huang, **"Dropedge: Towards deep graph convolutional networks on node classification,"** in *International Conference on Learning Representations*, 2020.

[22] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, **"SIGN: Scalable inception graph neural networks,"** *arXiv preprint arXiv:2004.11198*, 2020.

[23] **A. Cini**, I. Marisca, F. M. Bianchi, and C. Alippi, **"Scalable spatiotemporal graph neural networks,"** *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 6, pp. 7218–7226, Jun. 2023. DOI: 10.1609/aaai.v37i6.25880.

[24] A. Kazi, L. Cosmo, S.-A. Ahmadi, N. Navab, and M. M. Bronstein, **"Differentiable graph module (DGM) for graph convolutional networks,"** *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 2, pp. 1606–1617, 2022.

References vi

[25] **A. Cini**, D. Zambon, and C. Alippi, **"Sparse graph learning from spatiotemporal time series,"** *Journal of Machine Learning Research*, vol. 24, no. 242, pp. 1–36, 2023.

[26] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel, **"Neural relational inference for interacting systems,"** in *International conference on machine learning*, PMLR, 2018, pp. 2688–2697.

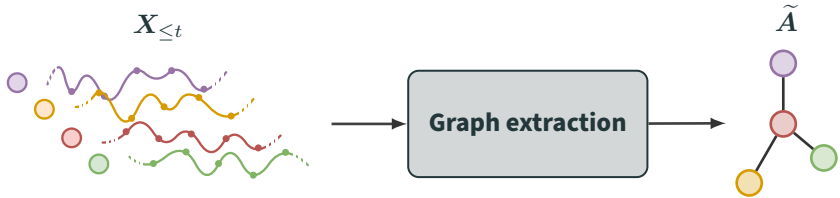
Bonus

Learning an adjacency matrix

☹ Relational information is not always available.

☺ Relational architectural biases can nonetheless be exploited

→ **extract a graph** from the time series or node attributes

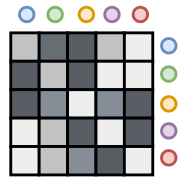


- When possible, the learned graph should be sparse.
- It can be interpreted as regularizing a spatial attention operator.

Time-series similarities

Probably, the simplest approach to extract a graph from the time series is by computing time series similarity scores.

- Pearson correlation
- Correntropy
- Granger causality
- Kernels for time series
- . . .



→ Thresholding might be necessary to obtain binary and sparse graphs.

End-to-end latent graph learning

An integrated approach: learn the graph end-to-end with the downstream task

$\rightarrow$ e.g., by minimizing the forecasting error (MAE, MSE...).

Two different formulations:

1. learning directly an adjacency matrix $\boldsymbol{A} \in \mathbb{R}^{N \times N}$;
2. learning a probability distribution over graphs p_Φ generating $\boldsymbol{A}$ (often $\in \{0, 1\}^{N \times N}$).

⚠ One key challenge is keeping both $\boldsymbol{A}$ and the subsequent computations sparse.

$\rightarrow$ non-trivial with gradient-based optimization.

Direct approach

A direct approach consists in learning $\widetilde{\boldsymbol{A}}$ as function $\xi(\,\cdot\,)$ of edge scores $\Phi \in \mathbb{R}^{N \times N}$ as

$$\widetilde{\boldsymbol{A}} = \xi\left(\Phi\right)$$

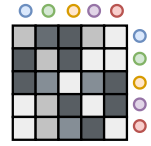
Edge scores Φ

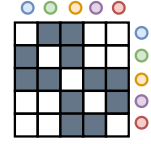
- can be a table of learnable model parameters,
- obtained as a function of the inputs and/or other parameters:

$$\Phi = \Phi(\boldsymbol{X}, \widetilde{\Phi}).$$

Function $\xi(\,\cdot\,)$ can enforce structures on $\widetilde{\boldsymbol{A}}$, like,

$\rightarrow$ make $\widetilde{\boldsymbol{A}}$ binary, a k-NN graph, a tree...

$\boldsymbol{\Phi} =$ 

$\widetilde{\boldsymbol{A}} =$ 

Edge score factorization

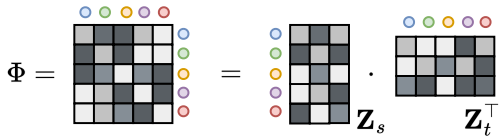
The number of possible edge scores is quadratic in the number of nodes ($\Phi \in \mathbb{R}^{N \times N}$)

$\rightarrow$ a common approach is to factorize Φ:

$$\widetilde{\boldsymbol{A}} = \xi(\Phi) \qquad \Phi = \boldsymbol{Z}_s \boldsymbol{Z}_t^\top$$

with

- $\boldsymbol{Z}_s \in \mathbb{R}^{N \times d}$ source node embeddings
- $\boldsymbol{Z}_t \in \mathbb{R}^{N \times d}$ target node embeddings



$\boldsymbol{Z}_s$ and $\boldsymbol{Z}_t$ can be learned as tables of (local) parameters or as a function of the input window.

[11] Wu *et al.*, "Graph wavenet for deep spatial-temporal graph modeling", IJCAI 2019.

Pro & Cons of the direct approach

- ☺ Easy to implement.
- ☺ Many possible parametrizations.
- ☺ Edge scores are usually easy to learn end-to-end.

- ☹ It often results in dense computations with $\mathcal{O}(N^2)$ complexity.
- ☹ Sparsifying $\boldsymbol{A}$ results in sparse gradients.
- ☹ Encoding prior structural information requires smart parametrizations.

Probabilistic methods

In this context, probabilistic methods aim at learning a parametric distribution p_Φ for $\boldsymbol{A}$.

- Different parametrizations of p_Φ allow for embedding graph structural priors on the sampled graphs, e.g., edge density, bound node degrees.

Independent edges

For every edge (i, j)

$$\boldsymbol{A}_{i,j} \sim \text{Bernoulli}(\sigma(\Phi_{i,j})).$$

Fixed-degree graphs

For each node i, sample w/o replacement k nodes from

$$\text{Categorical}\left(\text{SoftMax}(\Phi_{i,1}, \dots, \Phi_{i,N})\right).$$

- As seen before, Φ can be factorized and p_Φ made input dependent, e.g.,

$$\Phi = \xi\left(\boldsymbol{Z}_s \boldsymbol{Z}_t^\top\right), \qquad \boldsymbol{A} \sim p_\Phi\left(\boldsymbol{A} | \boldsymbol{X}_{<t}, \boldsymbol{U}_{<t}, \boldsymbol{V}\right).$$

[24] Kazi *et al.*, "Differentiable graph module (DGM) for graph convolutional networks", TPAMI 2022.

[25] **Cini** *et al.*, "Sparse graph learning from spatiotemporal time series", JMLR 2023.

Learning graph distributions

Training losses average over all graphs according to p_Φ, e.g., based on point predictions

$$\mathcal{L}(\Phi) = \mathbb{E}_{\boldsymbol{A}\sim p_\Phi}\left[\ell\left(\widehat{\boldsymbol{X}}_{t:t+H}, \boldsymbol{X}_{t:t+H}\right)\right], \qquad \mathcal{L}(\Phi) = \ell\left(\mathbb{E}_{\boldsymbol{A}\sim p_\Phi}\left[\widehat{\boldsymbol{X}}_{t:t+H}\right], \boldsymbol{X}_{t:t+H}\right),$$

where $\boldsymbol{X}_{t:t+H} = \mathcal{F}(\boldsymbol{X}_{t-W:t}, \boldsymbol{A}; \theta)$.

More generally, comparing predictive distributions by means of a divergence measure Δ

$$\mathcal{L}(\Phi) = \Delta\left(p_\Phi(\widehat{\boldsymbol{X}}_{t:t+H}), p(\boldsymbol{X}_{t:t+H})\right).$$

⚠ Gradient-based optimization requires computing $\nabla_\Phi \mathcal{L}(\Phi)$,
$\rightarrow$ i.e., differentiating w.r.t. the parameters of the integrated distribution.

- ☹ Analytical computations is often unfeasible;
- ☹ Monte Carlo approximations require care.

Monte Carlo gradient estimators

One approach is to reparametrize $\widetilde{\boldsymbol{A}} \sim p_\Phi(\boldsymbol{A})$ as: $\widetilde{\boldsymbol{A}} = g(\Phi, \varepsilon), \quad \varepsilon \sim p(\varepsilon)$
decoupling parameters Φ from the random component ε: $\nabla_\Phi \mathcal{L}(\Phi) = \mathbb{E}_\varepsilon \left[\nabla_\Phi \ell(\widehat{\boldsymbol{X}}, \boldsymbol{X}) \right]$.

- ☺ Practical and easy to implement,
- ☹ rely on continuous relaxations and make subsequent computations scale with $\mathcal{O}(N^2)$.

Conversely, score-function (SF) gradient estimators rely on the relation

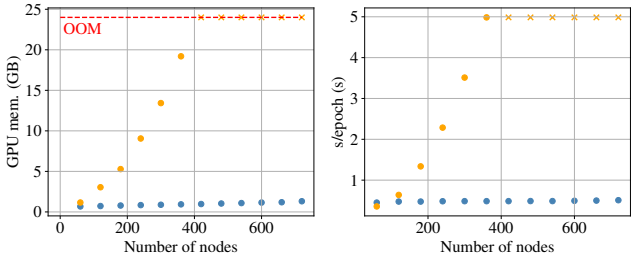
$$\nabla_\Phi \mathbb{E}_{p_\Phi} \left[\ell(\widehat{\boldsymbol{X}}, \boldsymbol{X}) \right] = \mathbb{E}_{p_\Phi} \left[\ell(\widehat{\boldsymbol{X}}, \boldsymbol{X}) \nabla_\Phi \log p_\Phi \right]$$

- ☹ suffer from high variance (use variance reduction techniques),
- ☺ allow to keep computations sparse through the model.

→ we can use Monte Carlo estimator.

[26] Kipf *et al.*, "Neural relational inference for interacting systems", ICML 2018.
[25] **Cini** *et al.*, "Sparse graph learning from spatiotemporal time series", JMLR 2023.

Computational efficiency



- Score-function
- Pathwise

With score-based gradient estimators $\nabla_\Phi \mathcal{L}(\Phi) = \mathbb{E}_{\boldsymbol{A}\sim p_\Phi}\left[\ell(\widehat{\boldsymbol{X}}, \boldsymbol{X})\nabla_\Phi \log p_\Phi(\boldsymbol{A})\right]$.

☺ They are computationally efficient as ∇_Φ is computed with respect to $\log p_\Phi(\boldsymbol{A})$.

- do not rely on continuous relaxation of discrete random variables;
- allow for sparse message passing to compute $\widehat{\boldsymbol{X}}$ (and, in turn, of $\ell(\widehat{\boldsymbol{X}}, \boldsymbol{X})$) by rely on sparse matrices $\boldsymbol{A}$.

☹ They can be sample inefficient due to the high variance of the gradient estimates.